
MASH: Modeling Abstention via Selective Help-Seeking

Mustafa Omer Gul¹ Claire Cardie¹ Tanya Goyal¹

Abstract

LLMs cannot reliably recognize their parametric knowledge boundaries and often hallucinate answers to outside-of-boundary questions. In this paper, we introduce MASH (Modeling Abstention via Selective Help-seeking), a training framework that readily extracts abstentions from LLMs. Our key idea is that any external help-seeking by an LLM, i.e. search tool use, can serve as a proxy for abstention if the external help (search) is appropriately penalized while also rewarding answer accuracy. MASH operationalizes this idea using reinforcement learning with a pay-per-search reward. We run experiments on three knowledge-intensive QA datasets. Our results show that MASH substantially improves upon the selective help-seeking performance of prior efficient search approaches; on multi-hop datasets, it improves answer accuracy by 7.6%. Furthermore, MASH demonstrates strong off-the-shelf abstention performance, showcasing behavior competitive with prior abstention methods that additionally require predetermining model knowledge boundaries to construct training data. Overall, we show MASH training effectively aligns search tool use with parametric knowledge, which can be successfully leveraged for making abstention decisions and efficient search tool use.²

1. Introduction

A reliable AI assistant should recognize its knowledge boundaries, what questions it can and cannot effectively respond to, and act accordingly when a question is outside its boundaries. Conventionally, LLMs learn their knowledge boundaries through alignment by explicitly training

for abstention (Yang et al., 2024; Cheng et al., 2024) and calibrated verbalization of uncertainty (Xu et al., 2024b; Stengel-Eskin et al., 2024). These strategies yield improved recognition of capability boundaries but are limited to reducing model errors. The number of questions a model can correctly answer remains unchanged. In this paper, we ask if we can design a training strategy that yields an abstention model capable of recognizing its boundaries, while learning techniques that expand its set of answerable questions?

We look at human behavior for inspiration. Humans recognize their limitations and can abstain when asked for knowledge they lack. Alternatively, they can seek outside help to answer these otherwise unanswerable questions. In this paper, we propose MASH (Modeling Abstention via Selective Help-seeking), a framework that indirectly trains LLMs for abstention by instead training models to engage in selective help-seeking, i.e. asking for help only when it cannot effectively respond to a query alone.

As a proof of concept, we explore this idea in the context of short-form question-answering tasks. We operationalize help-seeking as invoking a retrieval tool that returns information related to a given query. We train LLMs that selectively seek help (i.e. invoke retrieval) end-to-end with reinforcement learning using a penalty that discounts a correctness reward by the number of searches a model performs. An optimal policy optimizing this reward would, by definition, search only when a question cannot be reliably answered with parametric knowledge. In an inference mode with the same access to search, this model will mirror the above selective search behavior. More importantly, we can readily elicit abstention decisions from this same model by removing access to search tools. In that case, any search invocation serves as a proxy for abstention (see Figure 1). MASH, under this framing, effectively trains for two capabilities at the cost of one. Crucially, MASH assumes no privileged information regarding knowledge boundaries like standard abstention approaches (Yang et al., 2024; Cheng et al., 2024; Xu et al., 2024b) or require structured multi-agent interactions (Stengel-Eskin et al., 2024; Eisenstein et al., 2025).

We train MASH models using reinforcement learning with a pay-per-search reward (see Figure 1). However, baseline implementations of this idea (Wang et al., 2025a) result in efficient but sub-optimal search behaviors: models can

¹Department of Computer Science, Cornell University, Ithaca, NY, USA. Correspondence to: Mustafa Omer Gul <mog29@cornell.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

²Code and checkpoints are available at <https://github.com/momergul/mash>.

converge to always searching at least once. To address this, we propose a lightweight synthetic data curation and SFT pipeline that, crucially, assumes no information about knowledge boundaries. Instead, it serves to inject diverse, albeit parametrically unaligned, search behavior to improve exploration in RL training. Additionally, we extend reward formulations of prior work (Wang et al., 2025a) to obtain penalties with harsher levels of severity; we found this crucial for extracting good help-seeking behaviors via RL.

We run our experiments on 3 different knowledge-intensive datasets, and evaluate both the selective help-seeking performance with regular inference (with access to search) and abstention performance (without access to search). Our results show that MASH models substantially outperform previous efficient search baselines (Wang et al., 2025a) at balancing answer accuracy and searches. Notably, on multi-hop datasets, MASH reports a 7.6% accuracy improvement with a better distribution of searches. In fact, this performance is on par with search baselines (Jin et al., 2025) that allow any number of searches (up to a max value) without any penalty. We investigate this further and show that this improvement can be attributed to MASH showcasing a broader range of search strategies, i.e. diversity over number of searches, as a direct result of its training recipe.

Furthermore, we show that MASH reports strong off-the-shelf abstention performance. It achieves competitive performance with our strongest abstention baseline DPO (Rafailov et al., 2023; Cheng et al., 2024), which explicitly constructs a specialized training dataset for abstention training. Moreover, compared to prompting and supervised training methods for abstention (Yang et al., 2024), MASH reports higher answer accuracy (10 – 20% improvement) over non-abstained questions by better differentiating between answerable/unanswerable questions.

Taken together, our results demonstrate that MASH is an effective technique that yields an abstention model capable of recognizing its boundaries, while simultaneously expanding its set of answerable questions via help-seeking.

2. MASH: Modeling Abstention via Selective Help-seeking

2.1. Abstention Framework

Help-seeking LLMs We assume an inference setting where a language model π_θ can ask for help by sending a help request h to a helper $H(\cdot)$, which then returns a response $o \sim H(h)$. This helper H can take various forms: it could be a tool such as a retrieval model responding to a query, another stronger language model or an actual human in-the-loop. The model would then condition on the response o and continue its generation. Formally, given an input question q , the model

samples a trajectory $\tau \sim \pi_\theta(\cdot|q; H)$ of the form $\tau = (r_1, h_1, o_1, \dots, r_l, h_l, o_l, r_{l+1}, \hat{y})$, where each r_i represents reasoning, each h_i represents a help request generated by π_θ , o_i represents the associated output from helper $H(\cdot)$ and $\hat{y}$ represents the model’s final answer.

In this paper, we focus on knowledge-based domains. Here, h_i is a search query generated by π_θ , the helper $H(\cdot)$ is a retrieval model and o_i is a set of top- k documents retrieved by $H(h_i)$ from a document corpus. In practice, we assume that reasoning outputs r_i are enclosed between $\langle \text{think} \rangle$ and $\langle / \text{think} \rangle$, search queries between $\langle \text{search} \rangle$ and $\langle / \text{search} \rangle$, and answers between $\langle \text{answer} \rangle$ and $\langle / \text{answer} \rangle$ tokens. We use the terms help/search, and helper/retriever interchangeably.

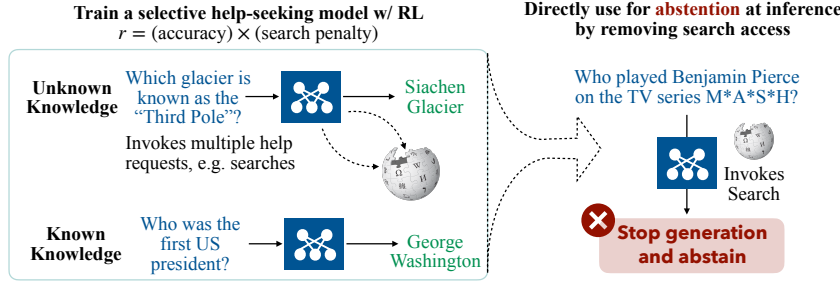
Training Objective We want the language model π_θ to recognize its knowledge boundaries. We posit that we can obtain such a model, without privileged information regarding parametric knowledge boundaries, by training the model to maximize its accuracy while minimizing the number of search requests. Specifically, we optimize the following proxy objective:

$$\max_{\theta} \mathbb{E}_{(q,y) \sim D, \tau \sim \pi_\theta(\cdot|q; H)} [r_{acc}(y, \tau) \cdot r_{help}(q, \tau)] \quad (1) \\ - \beta D_{KL}[\pi_\theta(\tau|q; H) || \pi_{\theta_{init}}(\tau|q; H)],$$

where D is the dataset, $r_{acc}(y, \tau) \in \{0, 1\}$ is a binary measure of correctness and $r_{help}(q, \tau) \in [0, 1]$ is a multiplicative penalty assigning a lower value the greater the number of searches in τ is. We use reinforcement learning, with the GRPO algorithm (Guo et al., 2025), for optimization.

Eliciting Abstention from a Selectively Help-Seeking Model Let π_{θ^*} be the optimal policy derived using the above objective. This model will selectively seek help as a function of its parametric accuracy for a given question q and the severity of the r_{help} penalty. Particularly, we prove in Appendix A that π_{θ^*} will always answer a question q parametrically, without help, if and only if its expected parametric accuracy for q is greater than the maximum attainable expected reward for q when seeking help, which relies on r_{help} . This is intuitive, as the optimal policy should choose to seek help for question q only if this resulted in an equivalent or higher reward than answering parametrically.

We re-frame the goal (and our subsequent evaluations) of our help-seeking model from efficiency, i.e. reducing number of searches, to parametric knowledge alignment, i.e. aligning search behavior with presence or absence of knowledge about a given question in the model’s parameters. Under this re-framing, we elicit abstentions from a selectively help-seeking model by treating any search invocation as a proxy for abstention. Following from our theoretical analysis in Appendix A, the optimal policy π_{θ^*} would abstain on a question q if and only if its expected parametric accuracy is



MASH: Modeling Abstention via Selective Help-seeking

Figure 1. Overview of MASH’s strategy for abstention. Help-seeking LLMs are RL-trained to maximize accuracy while minimizing searches. At inference, this same model is used for abstention by removing search access and treating any search requests as abstention.

less than or equal to the reward it would achieve if it were to seek help. Figure 1 illustrates this abstention framework, which we call MASH: Modeling Abstentions via Selective Help-seeking.

2.2. Training a Selective Help-Seeking Model

MASH training involves two main steps: (1) initializing θ_{init} in Equation 1 such that it displays diverse search behaviors (zero, one, or multiple searches) to encourage exploration, and (2) a reward function that appropriately balances accuracy and search tool penalty.

2.2.1. INITIALIZING π_θ W/ WARM-START SFT

RL training to optimize Equation 1 should, in theory, result in a model that selectively seeks help. However, in practice, we find that such training converges to sub-optimal policies that either exhibit degenerate strategies that always or never search, or that fail to learn to use the search tool effectively. In our work, we propose a **lightweight and model-agnostic synthetic data generation and finetuning pipeline** that results in a substantially better initial policy for subsequent RL training. Our data generation pipeline is designed to encourage diversity in the number of searches in model trajectories. Crucially, it requires no information about models’ parametric knowledge boundaries.

Synthetic data generation Our overall algorithm is a simple constrained decoding procedure. Let G be the synthetic data generation model. For each input question q in the warm-start training dataset, we first randomly sample a target number of searches $l \sim \{0, \dots, l_{max}\}$ to perform, where l_{max} is a hyperparameter controlling the maximum number of allowed searches. We then manually set the action tag (e.g., `<think>`, `<search>` or `<answer>`) the model should output at the start of each turn given the target number of searches and let the model perform generation normally otherwise. We sample N such trajectories per question, evaluate each and preferentially return a correct trajectory. Note that this constrained decoding process is

only used during synthetic data generation. Refer to Algorithm 2 in Appendix B.3 for more details.

A warm start SFT step is also included in recent works’ training pipelines to improve subsequent RL training (Guo et al., 2025; Gandhi et al., 2025; Wang et al., 2025b). However, we highlight one key difference. Contrary to prior works, our warm start process does not target correctness or alignment with models’ parametric knowledge, the two central goals of MASH. In fact, our synthetic data contains 35% errors with respect to answer correctness and, by design through the random sampling of search counts, yields a policy whose search behavior is unaligned with its parametric knowledge (discussed in Appendix B.3). We further mitigate any confounding factors by intentionally using a generator model G different from the model to be trained (as discussed in Section 3). The model learns how and when to use searches during RL training.

2.2.2. REWARD FORMULATION

Our reward $r(y, \tau)$ is a product of two terms: $r_{acc}(y, \tau)$, which is a binary correctness reward and $r_{help}(q, \tau)$, which is a search tool penalty. We compute $r_{acc}(y, \tau)$ using exact match. The form and severity of r_{help} will influence the learned help-seeking behavior. For input question q and G output trajectories $\{\tau_i\}_{i=1}^G$ sampled during GRPO, let n be the number of search queries in the most efficient and correct trajectory τ^{ef} and m be the number of queries in the given trajectory τ_i . We want r_{help} to appropriately penalize τ_i if $m > n$. There exists an arbitrarily high number of penalty formulations that satisfy this desiderata; we experiment with three:

1. **Exponential Decay**, defined as $r_{help}^{EXP}(q, \tau_i) = \lambda^{m-n}$ where λ controls the severity of the penalty.
2. **OTC** reward proposed by Wang et al. (2025a). We follow their recommendation and set c to the maximum number of

searches allowed in a single trajectory.

$$r_{help}^{OTC}(q, \tau_i) = \begin{cases} 1 & \text{if } m = n = 0 \\ \cos(\frac{m \cdot \pi}{2m+c}) & \text{if } n = 0 \\ \sin(\frac{m \cdot \pi}{m+n}) & \text{otherwise} \end{cases}, \quad (2)$$

3. OTC-Strict which enforces an extremely strict penalty when $m > n = 0$. Note that $n = 0$ indicates there is a correct trajectory τ^{ef} without searches. For these cases, any other trajectory τ_i that uses searches should get no reward under a very strict definition of answerability. Therefore, we set $r_{help}^{OTC-St}(q, \tau_i)$ to 0 for such cases. We can use any of the above two reward formulations for when $n > 0$, but choose OTC’s sinusoidal function to align with prior work.

3. Experimental Setup

Datasets and Models We run our experiments on three knowledge-intensive datasets: the single-hop dataset Natural Questions (NaturalQA) (Kwiatkowski et al., 2019), and multi-hop datasets HotPotQA (Yang et al., 2018) and 2WikiMultiHopQA (2Wiki) (Ho et al., 2020).³ We train and evaluate on each dataset separately; this allows us to evaluate MASH across tasks requiring different search strategies and with different distributions of parametrically answerable questions. Particularly, NaturalQA and HotPotQA differ in search strategies but have similar distributions of parametrically answerable questions, while 2Wiki is a multi-hop dataset containing a greater proportion of unanswerable questions than both (see Appendix B.4 for further analysis). We perform all training and evaluation on the Qwen2.5-3B base model (Qwen et al., 2025). We deliberately choose the base model over instruct as the latter has already undergone abstention training although the exact training strategy is unknown; we propose MASH as an alternative. We use the E5 retriever (Wang et al., 2022) and the 2018 Wikipedia dump as our knowledge source (Karpukhin et al., 2020).

Hyperparameters For the OTC reward, we follow Wang et al. (2025a) and set c equal to the maximum number of searches. For Exponential Decay, we set λ to 0.5 for Natural Questions and 0.8 otherwise, following hyperparameter tuning. We note that these hyperparameter choices imply the following decreasing order of severity of search penalty: OTC-STRICT→EXP→OTC. For each search query, we fix the response to be the top-3 retrieved passages and allow a maximum of 5 searches per trajectory. We use the veRL library (Sheng et al., 2025) for RL training. More training details are in Appendix B.1.

³We find that the “comparison” and “bridge-comparison” questions comprising in 2WikiMultiHopQA have unbalanced answer distributions (skewed towards “no”). This opens up the possibility of reward hacking by exploiting this dataset property. Therefore, we omit these questions from our training and evaluation.

Warm-start data generation We follow the strategy outlined in Section 2.2.1 to generate warm-start data for each dataset using Qwen2.5-32B base. Choosing a separate model ensures that information about knowledge boundaries is not baked into the SFT training data, while using the base variant avoids abstention behavior present in the instruct model from entering the training data and confounding results. For each dataset, we randomly sample 1000 questions from its training set and set $l_{max} = 2$. We select the trajectory for each question from $N = 5$ samples. Details can be found in Appendix B.3

We evaluate our selective help-seeking models in two inference modes: (1) **with access to search tools**, which directly aligns with its training, and (2) **without search tools**, where we use the help-seeking model for abstention. The baselines and evaluation metrics for these are described next.

3.1. Details for Inference Mode I: w/ Search Tools

Baselines We compare MASH’s help-seeking model against the following baselines that also conduct RL training, but with different setups: (1) **R1** trained using RL but without access to any search tools during training or evaluation. This baseline provides an upper bound for answer accuracy using only parametric knowledge. (2) **Search-R1** (Jin et al., 2025) trained with search tools and a binary correctness reward; showcasing an upper bound without any penalties for searching. (3) **OTC** (Wang et al., 2025a) RL-trained for efficient search tool use. We compare these baselines to three MASH variants that differ in reward penalties (refer to § 2.2.2). Note that MASH w/ OTC and OTC differ in the warm-start procedure applied to the former.

Evaluation Metrics We want our help-seeking model to strike a balance between answering parametrically (without search calls) and seeking help (with search calls). We report three metrics that collectively capture this: (1) **Accuracy (Acc)**, i.e. if the predicted answer matches the gold response. Due to the limitations of exact match, we use an LLM judge, namely DeepSeek-V3.1 (Liu et al., 2024), to determine this. (2) **Tool calls (TC)**, i.e. the average number of searches across trajectories. (3) **Tool Productivity (TP)** (Wang et al., 2025a), which is defined as $[\sum_{i=1}^{|\mathcal{D}|} \mathbb{I}\{y_i = \hat{y}_i\} / (1 + m_i)] / |\mathcal{D}|$ for test set $\mathcal{D}$. This discounts the accuracy of each output trajectory by its number of searches m_i . For all models, we report these metric averages over 4 samples. We use TP on the validation set to select our model checkpoints for all methods, except Search-R1 for which we use accuracy. TP results in a much inferior checkpoint selection for this case as early checkpoints where Search-R1 searches much less can end up achieving higher TP despite having poor accuracy.

Table 1. Accuracy, average number of tool calls (TC) and tool productivity (TP) for baselines and MASH evaluated under **inference w/ search tools**. MASH w/ OTC-ST is our best model with a 4.22% and 5.62% mean improvement on Acc and TP over baseline OTC.

Method	Natural Questions			HotPotQA			2Wiki		
	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑
R1	26.04	0.0	26.04	26.53	0.0	26.53	9.18	0.0	9.18
Search-R1 (Jin et al., 2025)	59.47	2.0	19.83	57.03	3.30	15.80	42.89	2.51	14.05
OTC (Wang et al., 2025a)	58.91	1.0	29.45	44.76	0.81	28.64	39.59	1.57	15.32
MASH w/ OTC	59.81	1.0	29.95	55.42	1.14	32.91	45.99	1.6	18.87
MASH w/ OTC-ST	56.37	0.64	38.63	53.32	1.10	32.55	46.23	1.64	19.08
MASH w/ EXP	54.35	0.65	36.61	53.78	1.07	32.09	44.32	1.53	18.10

3.2. Details for Inference Mode II: Abstention

In this mode, we follow the MASH process outlined in Figure 1 and § 2.1 to extract abstentions from a help-seeking model by removing access to search tools at inference.

Baselines We compare against the following baselines:

1. **5-shot prompting** with the base model, with abstention/not of in-context exemplars decided based on its parametric knowledge.
2. **Alignment for Honesty - Absolute (AFH-Abs)** (Yang et al., 2024), which does SFT on a specially curated abstention dataset by pairing each input question with either the output “I abstain” or the gold answer, depending on the base model’s knowledge boundaries.
3. **Alignment for Honesty - Multisample (AFH-Mult)** (Yang et al., 2024), which constructs multiple training samples for each question, pairing it with either “I abstain” or the gold answer depending on the average correctness over multiple outputs, for SFT training.
4. **DPO**, inspired by Cheng et al. (2024), which pairs each question with a preferred and dispreferred output. If the question is parametrically answerable, we set these to be the gold answer and “I abstain” respectively; this is switched for unanswerable questions. We train with the DPO loss objective (Rafailov et al., 2023) and SFT loss added as a regularizer (Pang et al., 2024).
5. **Ternary**, as considered by Xu et al. (2024a) and Wei et al. (2025), which conducts RL training with GRPO using a ternary reward assigning +1 for correct answers, 0 for abstention, and −1 abstention. We use the exact match reward from our other RL baselines to assess correctness.

Each of (1), (2) and (4) requires a definition of answerability; i.e. when we can claim that a question is answerable. A standard technique is to estimate the accuracy over 10 samples and use a threshold η to classify into answerable or not. However, there does not exist a consensus in prior works on how to decide this threshold (Yang et al., 2024; Cheng et al., 2024). In our paper, we follow (Yang et al., 2024) and set $\eta = 0.1$. Exact data curation and training details are in Appendix C.

Evaluation Metrics For abstention evaluation, we report two kinds of metrics: (1) **Answer Accuracy**: We report overall Accuracy, i.e. over the test set, and Precision, i.e. over non-abstained questions. Note that over-conservativeness, i.e. aggressively abstaining, will hurt overall accuracy but help precision, while under-conservativeness will cause the opposite. (2) **Abstention Classification**: This captures whether a model’s abstention behavior is aligned with its knowledge boundaries, agnostic of accuracy. To avoid defining answerability (different reward penalties assume a different threshold), we evaluate over two groups of questions unaffected by the choice of η : questions that the base models always answer incorrectly, with an average accuracy of 0, or always correctly, with an average accuracy of 1. Let %Abs(0) and %Abs(1) be the percentage of questions for which a model abstains for the above two groups, respectively. We report %Abs(0) and Delta (%Abs(0) − %Abs(1)). A model recognizing its knowledge boundaries should have a high abstention rate for always incorrect questions, i.e. %Abs(0), and a much lower rate for always correct questions, captured by a large margin %Abs(0) − %Abs(1). We do not evaluate on 2Wiki for abstention classification as there are only 58 test examples in the Abs(1) bucket, preventing reliable conclusions.

4. Results

4.1. Inference Mode I: w/ Search Tools

We first evaluate the performance of baselines and MASH in the inference setting with access to search tools. Table 1 reports overall answer accuracy, average tool calls and tool productivity for all methods. Additionally, we show the distribution of tool calls (TC=0/1/2+) and the corresponding accuracy per search count (subscript) in Table 2. This allows us to conduct an apples-to-apples comparison between models’ accuracy for the same number of tool calls.

MASH outperforms all search baselines on tool productivity by effectively balancing accuracy and searches. Our results in Table 1 show that MASH, particularly MASH w/ OTC-Strict, leads to a 5.62 point improvement

Table 2. Fine-grained tool use distribution (TC=0/1/2+ search) for baseline OTC and MASH. We also report answer accuracies for questions in each subset (subscript). For instance, MASH w/ OTC-Strict on 2Wiki answers questions with 2+ searches 77.5% of time, with an accuracy of 49.2%. MASH successfully off-loads questions to parametric answering (from TC=1 to TC=0) with minimal or no decrease in accuracy (HotPotQA & NaturalQA). Highlight intensity correlates with the proportion of questions answered with TC=0/1/2+.

Method	Natural Questions			HotPotQA			2Wiki		
	0	1	2+	0	1	2+	0	1	2+
OTC	0.0 _{0.0}	100.0 _{58.9}	0.0 _{0.0}	19.5 _{64.5}	80.2 _{40.0}	0.3 _{32.0}	3.1 _{24.1}	36.7 _{26.6}	60.2 _{48.3}
MASH w/ OTC	0.2 _{53.6}	99.8 _{59.8}	0.0 _{33.3}	23.5 _{66.5}	41.7 _{58.2}	34.8 _{44.6}	13.0 _{31.3}	13.9 _{35.9}	73.1 _{50.5}
MASH w/ OTC-ST	36.4 _{57.4}	63.5 _{55.8}	0.1 _{17.6}	28.9 _{59.9}	34.7 _{56.4}	36.4 _{45.2}	14.3 _{32.5}	8.3 _{42.3}	77.5 _{49.2}
MASH w/ EXP	35.2 _{53.6}	64.8 _{54.8}	0.0 _{20.0}	23.7 _{64.0}	45.5 _{53.4}	30.8 _{46.5}	11.8 _{32.2}	23.4 _{20.6}	64.9 _{55.1}

on tool productivity over baseline OTC on average across datasets. Surprisingly, MASH variants report accuracies on par with Search-R1 (trained without any tool use penalty) on multi-hop datasets HotPotQA and 2Wiki, but with a substantially lower number of searches (e.g. 1.10/1.64 vs 3.30/2.51). Moreover, this performance is a massive improvement over baseline OTC (8.56% and 6.64% improvements on HotPotQA and 2Wiki respectively) with only a slightly higher number of searches. Tool productivity, which accounts for both these metrics, improves by 3.83 points on average over baseline OTC. Together, these results suggest that MASH not only reduces the number of searches, but also better operationalizes them to maintain accuracy.

Severe search penalties are needed for parametric answers for single-hop NaturalQA. We observed that both baseline OTC and MASH with the lenient OTC penalty (MASH w/ OTC) do not learn to answer parametrically for NaturalQA, i.e. converge to TC=1 for all questions. On the other hand, MASH w/ OTC-Strict answers parametrically for 36% of the questions with only a 2.5% drop in accuracy, thereby improving tool productivity by 9 points. Similarly, MASH w/ Exp-Dec answers parametrically 35% of the time, with a 4.5% drop in accuracy⁴ compared to baseline OTC but a 7 point improvement in tool productivity.

The multi-hop datasets, HotPotQA and 2Wiki, report slightly higher average tool calls with the strictest penalty (MASH w/ OTC-Strict), presumably contradicting the above claim. However, fine-grained search distributions (see Table 2) show that, similarly to NaturalQA, OTC-Strict does answer parametrically (TC=0) more often than the lenient versions. The increase in average tools calls is due to a larger fraction of 2 searches.

MASH variants extract better and more diverse search behaviors for multi-hop datasets. Comparing search statistics for MASH w/ OTC and baseline OTC in Table 2, we see that they report a comparable number of parametric answers (23.5% vs 19.5%) but show very different search behaviors

⁴Note that MASH w/ Exp-Dec training did result in checkpoints with higher accuracies. However, we use tool productivity on the validation set as the metric to select the final checkpoint.

for the remaining questions. Particularly, the baseline OTC model without warm-start collapses to only one search for the remaining 80.2% of its trajectories, while the warm-started model (MASH w/ OTC) can perform a mixture of one and multi-hop searches. In fact, MASH variants report a much higher accuracy for one search questions (56.4% vs 40.0%) by offloading the more “difficult” questions, i.e. those the model cannot answer with only one search, to the two search bucket. Baseline OTC fails to do this and reports lower overall accuracy. We see similar trends for the other multi-hop dataset, 2Wiki, as well.

MASH successfully aligns search tool use with parametric knowledge. For NaturalQA, the fine-grained search statistics in Table 2 show that the questions MASH w/ OTC-Strict and w/ Exp answer parametrically have similar answer accuracy compared to those for which they invoke one search call (57.4 vs 55.8 for w/ OTC-Strict). This clearly shows that MASH can distinguish between parametrically answerable and not answerable questions and preferentially invoke tool calling for the latter to maintain overall accuracy.

4.2. Inference Mode II: w/ Abstention

MASH shows strong abstention behavior off-the-shelf. Table 3 (left) reports the answer accuracy for the overall test dataset (Acc) and the non-abstained questions (Prec) for each method.⁵ First, we observe that, apart from MASH w/ OTC on NaturalQA, all MASH variants substantially outperform the prompting and Alignment for Honesty based SFT approaches in terms of answer precision and report comparable overall accuracy. In a couple of instances, we find that the AFH (Absolute) baseline reports better accuracy (e.g. HotPotQA and NaturalQA) compared to MASH, but this is accompanied by a 10-20% drop in precision.

We find that MASH w/ OTC-Strict, our best performing model from Section 4.1, is comparable to DPO for NaturalQA and HotPotQA; it outperforms DPO based on Prec. (59.9 vs 53.1 for HotPotQA) but reports lower overall ac-

⁵Note that it is possible to game one of these metrics by being over- or under-conservative. Therefore, all our conclusions are based on analyzing the two metrics collectively.

curacy (17.3 vs 19.9). We attribute this to MASH w/ OTC-Strict being more conservative (i.e. more likely to abstain) than DPO. For 2Wiki, MASH w/ OTC-Strict outperforms DPO on both Acc and Prec. This competitive performance is despite MASH using the noisier exact match as the reward signal (due to our compute constraints), while DPO uses a stronger and more reliable LLM judge (DeepSeek-V3.1).

Finally, the Ternary baseline is outperformed by all methods and converges to always abstaining within 25 training steps on all datasets. This reproduces observations by Wei et al. (2025), which similarly found convergence to abstaining for a vast majority of questions with exact match as the correctness signal. Using a stronger LLM judge alternative is outside the scope of this paper due to resource limitations.

MASH can differentiate between answerable and unanswerable questions. Table 3 (right) shows the abstention classification results. As expected, we find that DPO models explicitly trained for abstention report the best results. Encouragingly, we see that MASH variants, except MASH w/ OTC on NaturalQA which does not learn to answer parametrically, report similarly high Abs(0) percentages as DPO. While DPO reports higher Delta for both datasets, Table 3 shows that these large improvements in Delta are often accompanied by a drop in precision. For example, DPO reports 13.2% better Delta than MASH w/ OTC-Strict for HotPotQA, but reports a 6.8% lower precision.

Taken together, these results present an encouraging picture for the idea of modeling abstention with models trained for the auxiliary selective help-seeking task. They show that although MASH does not train explicitly for abstention, its **abstention behavior is analogous to that of abstention methods leveraging oracle information regarding model knowledge boundaries.**

4.3. Generalization to Other Model Scales and Families

We repeat experiments on Qwen2.5-7B base and Qwen3-4B base to determine whether MASH’s success generalizes to different parameter scales and model families, respectively. Due to the cost of replicating all RL experiments, we restrict our attention to HotPotQA and conduct RL training only for the OTC baseline and for MASH w/ OTC-Strict, the best performing variant for Qwen2.5-3B base.

MASH continues to outperform baselines at different model scales and families. Under the inference setting with search enabled, the OTC baseline converges to searching once for both models. MASH w/ OTC-Strict, on the other hand, consistently performs a mixture of parametric answers, and one- and multi-hop searches. Furthermore, it achieves average improvements of 2.98% and 9.59 on accuracy and tool productivity. Similar trends persist for abstention. MASH w/ OTC-Strict outperforms prompting and Align-

ment for Honesty. MASH yet again displays competitive performance with DPO, with different accuracy-precision trade-offs. Particularly, Qwen2.5-7B-Base achieves superior precision (60.67% vs 48.25%), while Qwen3-4B-Base achieves superior accuracy (20.96% vs 16.72%) over DPO. Result tables can be found in Appendix D.

4.4. Impact of Warm-start on Performance

The comparative results of the OTC baseline and MASH w/ OTC in both Tables 1 and 2 indicate that the warm-start SFT training is key to MASH’s success. By design, it enables the model to explore diverse trajectories with varying numbers of search tool calls during RL. Here, we study the impact of warm start for all reward formulations. Table 4 reports the performance for all three without warm start (refer to Table 1 for comparison with models trained with warm start).

Warm-start adds stability to harsher penalties. The OTC reward shows the best help-seeking behavior when considering all datasets collectively. However, we discussed in § 4.1 that the search behavior with warm-start is far superior to without for OTC. Recall that Exponential Decay and OTC-Strict both impose harsher penalties on search tool use than OTC. We observe that this results in severe training instabilities for these two when trained without warm-start – the HotPotQA policy collapses to zero searches for OTC-Strict and the 2Wiki policy collapses for both Exponential Decay and OTC-Strict. Warm-start SFT, however, enables both to have successful training runs on all datasets, with OTC-Strict with warm-start even substantially outperforming OTC on all datasets for abstention metrics.

4.5. Do Oracle Helpers Improve Selective Help-seeking?

All experiments in Section 4 rely on a retrieval model (E5; (Wang et al., 2022)) as the helper $H(\cdot)$. However, search results output by these retrievers can be noisy, which in turn generates a noisy signal for training the selective help-seeking LLM via RL. This prompts us to investigate if improving the “helper”, as opposed to the reward or initialization, can improve the learned help-seeking behavior.

Setup: We set $H(\cdot)$ to be an oracle; it directly returns the gold answer if the LLM invokes a help tag in its trajectory (exact prompts used is included in Appendix E). We train all MASH variants (OTC, OTC-Strict, Exp) for all datasets. Warm-start training is done for each individually with $l_{max} = 1$.

Results: Training with oracle helpers fails to yield selective help-seeking. We find that each training run converged to always asking for help within the first 50 training steps, even with stricter penalties. Note that the optimal policy should display selective help-seeking (answer parametrically for known questions) to maximize reward. However,

Table 3. Answer accuracy and abstention classification results for specialized abstention approaches and MASH. For answer accuracy, we report Acc over the entire test set and Prec, the accuracy over non-abstained answers. For classification, we report Abs(0), the abstention rate for unanswerable questions (higher better), and Delta, the margin between abstention rates for unanswerable and answerable questions (higher better). We highlight runs with similar performance but different trade-offs between Acc and Prec or Abs(0) and Delta in gray.

Method	Answer Accuracy						Abstention Classification			
	NaturalQA		HotPotQA		2Wiki		NaturalQA		HotPotQA	
	Acc	Prec	Acc	Prec	Acc	Prec	Abs(0) ↑	Delta ↑	Abs(0) ↑	Delta ↑
OTC	0.0	–	12.6	64.5	0.75	24.1	100.0	0.0	95.3	41.4
MASH w/ OTC	0.1	53.6	15.6	66.5	4.1	31.3	99.9	0.1	94.8	52.3
MASH w/ OTC-ST	20.9	57.4	17.3	59.9	4.6	32.5	85.4	66.1	91.2	60.3
MASH w/ EXP	18.9	53.6	15.2	64.0	3.8	32.2	85.6	62.7	94.5	52.7
5-shot Prompting	23.4	42.5	14.7	31.5	3.6	10.9	60.2	44.6	60.5	26.9
AFH (Absolute)	21.7	43.3	20.7	34.2	4.7	18.5	67.7	48.1	50.4	35.4
AFH (Multisample)	14.7	54.8	12.9	53.8	2.6	29.2	87.9	52.1	89.2	57.6
DPO	22.3	56.2	19.9	53.1	3.3	31.6	84.5	71.6	85.9	73.5
Ternary	0.0	–	0.0	–	0.0	–	100.0	0.0	100.0	0.0

Table 4. MASH w/o warm-start tested with search enabled.

Method	Natural Questions			HotPotQA			2Wiki		
	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑
OTC	58.91	1.0	29.45	44.76	0.81	28.64	39.59	1.57	15.32
OTC-ST	52.35	0.49	39.28	26.97	0.0	26.97	10.41	0.0	10.41
EXP	57.59	1.00	28.80	41.46	0.71	28.67	9.72	0.0	9.72

Table 5. Out-of-distribution accuracy (w/ and w/o search) and abstention classification results for baseline OTC, best MASH, and best abstention models trained on HotPotQA .

Method	Acc↑	Acc w/ tool↑	Abs(0) ↑	Delta↑
Natural Questions				
OTC	2.09	54.35	99.04	8.32
MASH w/ OTC-ST	18.25	51.24	79.95	51.63
DPO	24.4	-	77.38	68.23
TriviaQA				
OTC	4.07	71.43	96.95	7.11
MASH w/ OTC-ST	30.52	67.61	77.53	51.55
DPO	41.6	-	71.71	66.23

we do not observe this in practice, as always seeking help is an easy strategy to discover. For OTC and Exponential Decay, it is given non-zero rewards for all inputs. For OTC-Strict, it is given a positive reward for each question without correct parametric answers, which is be common early in training. As such, the noisiness of the retrieval model is crucial to extract selective help-seeking in a manner aligned with its parametric knowledge. We require RL algorithms with better exploration to succeed in this oracle setting.

4.6. Out-of-Distribution Performance

Finally, we evaluate our trained models out-of-distribution. Due to space, we restrict our analysis to the OTC baseline, and our best performing MASH variant w/ OTC-Strict and

the best abstention baseline (DPO) trained on HotPotQA. We evaluate generalization to other training datasets and an additional single-hop dataset TriviaQA (Joshi et al., 2017).

Results: Table 5 reports our results (NaturalQA and 2Wiki models are in Appendix G). MASH generalizes better than the OTC (higher Accuracy and Delta values), which abstains on nearly all questions out-of-distribution. MASH also reports better Abs(0) performance than DPO, but lower Delta. We attribute this to MASH generalizing more conservatively out-of-domain. With 2Wiki, which only contains multi-hop questions, MASH generalizes well to HotPotQA but performs poorer on single-hop datasets. We argue that, for out-of-distribution generalization, abstention and invoking search tools is the more ideal decision. With search enabled, our HotPotQA-trained MASH model attains 26.43% higher accuracy than DPO, which is limited to abstention.

5. Related Work

Abstention and Verbalized Uncertainty As LLM adoption increases, it is critical that models recognize their capability boundaries and faithfully report their uncertainty. Past work has explored this problem from many directions, developing techniques for hallucination detection (Du et al., 2024; Chen et al., 2024), abstention (Yang et al., 2024; Cheng et al., 2024) and calibration (Kapoor et al., 2024), with methods ranging from model prompting (Feng et al., 2024) and hidden state probing (Du et al., 2024; Chen et al., 2024) to training of the model itself (Kadavath et al., 2022). We focus on methods for abstention that train a model that intrinsically recognizes its knowledge boundaries.

Past work predominantly relies on pipelined approaches that first estimate a model’s knowledge boundaries and then use this information either to construct datasets to be used for SFT (Yang et al., 2024; Zhang et al., 2024) and DPO train-

ing (Cheng et al., 2024), or to train model-specific reward functions for RLHF (Xu et al., 2024a). Some work additionally seeks to align models to explicitly verbalize their uncertainty, either by explicitly constructing SFT data that summarizes uncertainty over multiple samples (Xu et al., 2024b) or through structured, multi-agent interaction scenarios (Stengel-Eskin et al., 2024; Eisenstein et al., 2025). The work closest to ours is the Collaborative Self-Play approach of Eisenstein et al. (2025), which likewise investigates learning knowledge boundaries in conjunction with tool use. However, their approach relies on a specialized multi-agent scenario where each agent has a constrained set of actions and is restricted to single-hop question-answering settings. In contrast, MASH demonstrates that models can effectively learn their knowledge boundaries through the mere optimization of an efficient tool-use reward, without requiring any task-specific multi-agent scaffolding or assumptions on task complexity.

Selective RAG There has separately been much attention on developing methods for determining when to search or continue searching in the context of retrieval augmented generation (RAG) approaches. Such methods are predominantly specialized towards the RAG setting, with methods influencing both when search is performed and the content of the search queries themselves. Approaches commonly involve estimating uncertainty, be it through operations on hidden model states (Yao et al., 2025; Baek et al., 2025), self-consistency over samples (Ding et al., 2025) or output probabilities (Jiang et al., 2023; Su et al., 2024). While we focus on knowledge-intensive queries, our approach is task-agnostic and only involves end-to-end reinforcement learning optimization with an efficiency reward.

Augmenting LLMs with Tool-Use There has long been interest in leveraging tool-use to augment LLM capabilities (Schick et al., 2023; Yao et al., 2023), with post-training pipelines for foundation models (Yang et al., 2025; Team et al., 2025) increasingly featuring dedicated training for tool-use. We build on top of recent work training LLMs to use search tools with RL (Jin et al., 2025), particularly on top of the OTC reward formulation of Wang et al. (2025a). While OTC provides us one of our reward formulations, Wang et al. (2025a) exclusively analyze gains in efficiency without reference to parametric knowledge and, furthermore, fail to answer parametrically on their experiments involving Natural Questions. We instead propose MASH as an alternative framework for abstention, provide analyses with varying reward formulations, consistently produce a mixture between parametric answers and search, and provide explicit comparisons against abstention methods.

6. Conclusion

We propose MASH, a novel framework that trains LLMs for selective help-seeking, and readily extracting abstention behaviors. MASH trains models for two capabilities at the cost of one – models learn how to use search tools and synthesize information, and distinguish between answerable/unanswerable questions. Our results on 3 short-form knowledge-intensive datasets show that MASH outperforms previous efficient search baselines on overall accuracy when allowed searches and also demonstrates strong abstention behaviors, analogous to specialized abstention methods.

Acknowledgments

This project was partially supported by NSF grant IIS-2433072, and a gift from Google. We gratefully acknowledge use of the research computing resources of the Empire AI Consortium, Inc, with support from Empire State Development of the State of New York, the Simons Foundation, and the Secunda Family Foundation. We thank the Cornell NLP group for helpful discussions and comments.

Impact Statement

We expect MASH to have positive consequences, like other methods focused on improving the abstention capabilities of LLMs, thanks to its goal of aligning model behavior with its knowledge boundaries and mitigating hallucinations. We do not expect harm to emerge from the application of MASH, beyond that which comes from any work whose goal is to advance the development of large language models.

References

- Baek, I., Chang, H., Kim, B., Lee, J., and Lee, H. Probing-RAG: Self-probing to guide language models in selective document retrieval. In Chiruzzo, L., Ritter, A., and Wang, L. (eds.), *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 3287–3304, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.181. URL <https://aclanthology.org/2025.findings-naacl.181/>.
- Chen, C., Liu, K., Chen, Z., Gu, Y., Wu, Y., Tao, M., Fu, Z., and Ye, J. INSIDE: LLMs’ internal states retain the power of hallucination detection. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Zj12nz1Qbz>.
- Cheng, Q., Sun, T., Liu, X., Zhang, W., Yin, Z., Li, S., Li, L., He, Z., Chen, K., and Qiu, X. Can AI assistants

- know what they don't know? In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=girxGkdECL>.
- Ding, H., Pang, L., Wei, Z., Shen, H., and Cheng, X. Rowen: Adaptive retrieval-augmented generation for hallucination mitigation in llms. In *Proceedings of the 2025 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, SIGIR-AP 2025*, pp. 12–21, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400722189. doi: 10.1145/3767695.3769500. URL <https://doi.org/10.1145/3767695.3769500>.
- Du, X., Xiao, C., and Li, S. Haloscope: Harnessing unlabeled llm generations for hallucination detection. *Advances in Neural Information Processing Systems*, 37: 102948–102972, 2024.
- Eisenstein, J., Aghajani, R., Fisch, A., Dua, D., Huot, F., Lapata, M., Zayats, V., and Berant, J. Don't lie to your friends: Learning what you know from collaborative self-play. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=2vDJiGUfhV>.
- Feng, S., Shi, W., Wang, Y., Ding, W., Balachandran, V., and Tsvetkov, Y. Don't hallucinate, abstain: Identifying LLM knowledge gaps via multi-LLM collaboration. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14664–14690, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.786>.
- Gandhi, K., Chakravarthy, A. K., Singh, A., Lile, N., and Goodman, N. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective STars. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=QGJ9ttXLty>.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., Li, H., McDonnell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- Gekhman, Z., Yona, G., Aharoni, R., Eyal, M., Feder, A., Reichart, R., and Herzig, J. Does fine-tuning LLMs on new knowledge encourage hallucinations? In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 7765–7784, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.444. URL <https://aclanthology.org/2024.emnlp-main.444/>.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- Ho, X., Duong Nguyen, A.-K., Sugawara, S., and Aizawa, A. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In Scott, D., Bel, N., and Zong, C. (eds.), *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6609–6625, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.580. URL <https://aclanthology.org/2020.coling-main.580/>.
- Jiang, Z., Xu, F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. Active retrieval augmented generation. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7969–7992, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.495. URL <https://aclanthology.org/2023.emnlp-main.495/>.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S. O., Wang, D., Zamani, H., and Han, J. Search-r1: Training LLMs to reason and leverage search engines with reinforcement learning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=Rwhi91lideu>.
- Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Barzilay, R. and Kan, M.-Y. (eds.), *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, Vancouver,

- Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147/>.
- Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., Ganguli, D., Hernandez, D., Jacobson, J., Kernion, J., Kravec, S., Lovitt, L., Ndousse, K., Olsson, C., Ringer, S., Amodei, D., Brown, T., Clark, J., Joseph, N., Mann, B., McCandlish, S., Olah, C., and Kaplan, J. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- Kapoor, S., Gruver, N., Roberts, M., Collins, K., Pal, A., Bhatt, U., Weller, A., Dooley, S., Goldblum, M., and Wilson, A. Large language models must be taught to know what they don’t know. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 85932–85972. Curran Associates, Inc., 2024. doi: 10.52202/079017-2729.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. Dense passage retrieval for open-domain question answering. In Weber, B., Cohn, T., He, Y., and Liu, Y. (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.550. URL <https://aclanthology.org/2020.emnlp-main.550/>.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Lin, S.-C., Gao, L., Oguz, B., Xiong, W., Lin, J., Yih, W.-t., and Chen, X. Flame: Factuality-aware alignment for large language models. *Advances in Neural Information Processing Systems*, 37:115588–115614, 2024.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding rl-zero-like training: A critical perspective. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=5PAF7PAY2Y>.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. s1: Simple test-time scaling. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20275–20321, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1025. URL <https://aclanthology.org/2025.emnlp-main.1025/>.
- Pang, R. Y., Yuan, W., Cho, K., He, H., Sukhbaatar, S., and Weston, J. Iterative reasoning preference optimization. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, pp. 116617–116637, 2024.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741, 2023.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys ’25*, pp. 1279–1297, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711961. doi: 10.1145/3689031.3696075. URL <https://doi.org/10.1145/3689031.3696075>.
- Stengel-Eskin, E., Hase, P., and Bansal, M. LACIE: Listener-aware finetuning for calibration in large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=RnvgYd9RAh>.

- Su, W., Tang, Y., Ai, Q., Wu, Z., and Liu, Y. DRA-GIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12991–13013, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.702. URL <https://aclanthology.org/2024.acl-long.702/>.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T., Lambert, N., Huang, S., Rasul, K., and Gallouédec, Q. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
- Wang, H., Qian, C., Zhong, W., Chen, X., Qiu, J., Huang, S., Jin, B., Wang, M., Wong, K.-F., and Ji, H. Acting less is reasoning more! teaching model to act efficiently. *arXiv preprint arXiv:2504.14870*, 2025a.
- Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., and Wei, F. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- Wang, Z., Zhou, F., Li, X., and Liu, P. Octothinker: Mid-training incentivizes reinforcement learning scaling. *arXiv preprint arXiv:2506.20512*, 2025b.
- Wei, Z., Yang, X., Sun, K., Wang, J., Shao, R., Chen, S., Kachuee, M., Gollapudi, T., Liao, T., Scheffer, N., et al. Truthrl: Incentivizing truthful llms via reinforcement learning. *arXiv preprint arXiv:2509.25760*, 2025.
- Xu, H., Zhu, Z., Zhang, S., Ma, D., Fan, S., Chen, L., and Yu, K. Rejection improves reliability: Training LLMs to refuse unknown questions using RL from knowledge feedback. In *First Conference on Language Modeling*, 2024a. URL <https://openreview.net/forum?id=lJMioZBoR8>.
- Xu, T., Wu, S., Diao, S., Liu, X., Wang, X., Chen, Y., and Gao, J. SaySelf: Teaching LLMs to express confidence with self-reflective rationales. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 5985–5998, Miami, Florida, USA, November 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.343. URL <https://aclanthology.org/2024.emnlp-main.343/>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, Y., Chern, E., Qiu, X., Neubig, G., and Liu, P. Alignment for honesty. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=67K3Xlvw8L>.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Riloff, E., Chiang, D., Hockenmaier, J., and Tsujii, J. (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL <https://aclanthology.org/D18-1259/>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Yao, Z., Qi, W., Pan, L., Cao, S., Hu, L., Weichuan, L., Hou, L., and Li, J. SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27022–27043, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1312. URL <https://aclanthology.org/2025.acl-long.1312/>.
- Zhang, H., Diao, S., Lin, Y., Fung, Y., Lian, Q., Wang, X., Chen, Y., Ji, H., and Zhang, T. R-tuning: Instructing large language models to say ‘I don’t know’. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7113–7139, Mexico City, Mexico, June 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.naacl-long.394>.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

A. Theoretical Analysis

A.1. Preliminaries

In this section, we provide a theoretical analysis of the search behavior the optimal policy for the proxy objective of Equation 1 will display for a given question. Specifically, we will formally demonstrate that the optimal policy must produce a parametric answer to a question q if and only if the maximum achievable expected accuracy when answering q parametrically is greater than the maximum achievable expected reward when performing searches.

Before starting the analysis proper, we will first make two assumptions:

- **Assumption 1:** We set the KL penalty weight $\beta = 0.0$ to simplify the proxy objective and focus solely on the search behavior of an optimal policy maximizing reward.
- **Assumption 2:** We assume that the optimal policy cannot achieve perfect expected accuracy (i.e., average accuracy across multiple samples), for all questions when answering parametrically. In other words, the maximum achievable expected accuracy is bounded for some questions. This follows by definition from our training setup, where an LLM cannot acquire new knowledge beyond the base model’s knowledge boundaries during RL and from prior work, which suggests that standard finetuning approaches on new knowledge risk an increase in hallucinations rather than incorporation of knowledge (Lin et al., 2024; Gekhman et al., 2024).

We then make three definitions to be used in our proofs:

- **Definition 1:** Let $N_s(\tau)$ be the number of search calls made by a trajectory τ .
- **Definition 2:** Let $r_{acc}(y, \tau) \in \{0, 1\}$ and $r_{help}(q, \tau)$ be defined such that $r_{acc}(y, \tau) \cdot r_{help}(q, \tau) = 1$ if $r_{acc}(y, \tau) = 1$ and $N_s(\tau) = 0$. This follows from the definitions set in Section 2.1.
- **Definition 3:** Consider a question-answer pair (q, y) . Then, we define:

$$R_i^*(q, y) = \max_{\theta} E_{\tau \sim \pi_{\theta}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = i]$$

as the maximum achievable expected reward by a policy when performing i searches.

A.2. Proof

Lemma 1: For a given question-answer pair (q, y) , the optimal policy π_{θ^*} must answer the question q parametrically (without any searches) if and only if $R_0^*(q, y) > R_i^*(q, y)$ for all $i > 0$.

Proof of Lemma 1: We firstly note that the expected reward for any arbitrary policy θ given a question-answer pair (q, y) can be written as a weighted sum of its expected reward when answering question q with different search counts. We show this below:

$$\begin{aligned} E_{\tau \sim \pi_{\theta}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau)] &= \sum_{\tau} [\pi_{\theta}(\tau|q, H) \cdot r_{acc}(y, \tau) \cdot r_{help}(q, \tau)] \\ &= \sum_{i=0}^{\infty} \sum_{\tau|N_s(\tau)=i} [\pi_{\theta}(\tau|q, H) \cdot r_{acc}(y, \tau) \cdot r_{help}(q, \tau)] \\ &= \sum_{i=0}^{\infty} \left[P(N_s(\tau) = i|q; \theta) \cdot \sum_{\tau|N_s(\tau)=i} \left[\frac{\pi_{\theta}(\tau|q, H)}{P(N_s(\tau) = i|q; \theta)} \cdot r_{acc}(y, \tau) \cdot r_{help}(q, \tau) \right] \right] \\ &= \sum_{i=0}^{\infty} [P(N_s(\tau) = i|q; \theta) \cdot E_{\tau \sim \pi_{\theta}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = i]], \end{aligned}$$

where $P(N_s(\tau) = i|q; \theta)$ is the probability that a policy parameterized by θ will produce a trajectory τ with $N_s(\tau) = i$ given question q .

Given this, we will first prove the forward direction. Let θ^* be the parameters optimizing the objective and let (q, y) be an arbitrary question-answer pair. Assume that π_{θ^*} must answer question q parametrically. This firstly implies that $P(N_s(\tau) = 0|q; \theta^*) = 1$ and, due to the optimality of θ^* , that

$$\begin{aligned} E_{\tau \sim \pi_{\theta^*}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = 0] &= \max_{\theta} E_{\tau \sim \pi_{\theta}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = 0] \\ &= R_0^*(q, y). \end{aligned}$$

Furthermore, we note that the fact that θ^* must answer q parametrically implies that

$$R_0^*(q, y) = E_{\tau \sim \pi_{\theta^*}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = 0] > R_i^*(q, y)$$

for all $i > 0$, where $R_i^*(q, y)$ is the maximum attainable expected reward when using i searches. Assume for the sake of contradiction this were not true and that there existed some i such that

$$R_0^*(q, y) = E_{\tau \sim \pi_{\theta^*}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = 0] \leq R_i^*(q, y).$$

Let $\hat{\theta}$ be a policy that sets $P(N_s(\tau) = i|q; \hat{\theta}) = 1$ and displays the same behavior as θ^* all other questions. In the first case, $R_0^*(q, y) = R_i^*(q, y)$. Here, $\hat{\theta}$ would have the same expected reward as θ^* and be another optimal policy. However, this contradicts our assumption that the optimal policy *must* answer q parametrically. In the other case, $R_0^*(q, y) < R_i^*(q, y)$. In this instance, $\hat{\theta}$ achieves a greater expected reward than θ^* , which contradicts the assumption that θ^* is optimal. Hence, $R_0^*(q, y) > R_i^*(q, y)$ for all $i > 0$, which proves the forward direction.

We will next prove the backwards direction. Let θ^* be the parameters optimizing the objective and assume that $R_0^*(q, y) > R_i^*(q, y)$. We claim that θ^* must answer q parametrically. Let C be an arbitrary set of non-negative integers containing at least one positive integer $i > 0$ and assume for the sake of contradiction that θ^* set $P(N_s(\tau) = i|q; \theta^*) > 0$ for all $i \in C$ and $\sum_{i \in C} P(N_s(\tau) = i|q; \theta^*) = 1$. Then, it firstly follows that:

$$\begin{aligned} E_{\tau \sim \pi_{\theta^*}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau)] &= \sum_{i \in C} P(N_s(\tau) = i|q; \theta^*) E_{\tau \sim \pi_{\theta^*}(\cdot|q, H)}[r_{acc}(y, \tau) \cdot r_{help}(q, \tau) | N_s(\tau) = i] \\ &= \sum_{i \in C} P(N_s(\tau) = i|q; \theta^*) R_i^*(q, y), \end{aligned}$$

due to our assumption that θ^* is optimal. However, due to our assumption that $R_0^*(q, y) > R_i^*(q, y)$ for all $i > 0$, we also have that

$$\sum_{i \in C} P(N_s(\tau) = i|q; \theta^*) R_i^*(q, y) < \sum_{i \in C} P(N_s(\tau) = i|q; \theta^*) R_0^*(q, y) = R_0^*(q, y).$$

Given this, we can construct an alternative policy $\hat{\theta}$ that sets $P(N_s(\tau) = 0|q; \hat{\theta}) = 1$ and behaves the same as θ^* on all other questions. This policy $\hat{\theta}$ will have a larger expected reward than θ^* , which contradicts our assumption that θ^* is optimal. Hence, θ^* cannot set $P(N_s(\tau) = i|q; \theta^*) > 0$ for any $i > 0$ and *must* answer q parametrically. This proves the reverse direction. As a result, our claim holds.

Lemma 2: Let (q, y) be an arbitrary question-answer pair. Then, the maximum achievable expected accuracy when answering q parametrically is equal to $R_0^*(q, y)$.

Proof of Lemma 2: Consider an arbitrary trajectory τ such that $N_s(\tau) = 0$. We claim that the accuracy of τ is equal to the reward of τ . In the first possibility, $r_{acc}(y, \tau) = 1$. Then, that the reward $r_{acc}(y, \tau) \cdot r_{help}(q, \tau) = 1$ follows from Definition 2. In the second possibility, $r_{acc}(y, \tau) = 0$. Then, the reward

$$r_{acc}(y, \tau) \cdot r_{help}(q, \tau) = 0 \cdot r_{help}(q, \tau) = 0.$$

Hence, the accuracy and reward of τ are equal when $N_s(\tau) = 0$. This further implies that the maximum achievable expected accuracy is equal to the maximum achievable expected reward when answer q parametrically. This latter quantity, by Definition 3, is $R_0^*(q, y)$. The lemma is therefore correct.

Theorem: For a given question-answer pair (q, y) , the optimal policy π_{θ^*} must answer the question q parametrically (without any searches) if and only if the maximum achievable expected accuracy when answering q parametrically is greater than the maximum achievable expected reward when performing searches.

Proof of Theorem: Consider an arbitrary question-answer pair (q, y) and let θ^* be the optimal policy. Then, Lemma 1 implies that θ^* must answer the question q parametrically if and only if $R_0^*(q, y) > \max_{i>0} R_i^*(q, y)$. Furthermore, by Lemma 2, we know that the maximum achievable expected accuracy when answering q is equal to $R_0^*(q, y)$. These two lemmas together imply that the optimal policy π_{θ^*} must answer the question q parametrically if and only if the maximum achievable expected accuracy when answering q parametrically is greater than the maximum achievable expected reward when performing searches. The theorem holds.

Table 6. Accuracy, average number of tool calls (TC) and tool productivity (TP) for warm-start initialization and the best-performing MASH variant evaluated under **inference w/ search tools**. The warm-start initialization obtains poorer accuracy and tool-productivity scores than the RL-trained MASH model in all cases.

Method	Natural Questions			HotPotQA			2Wiki		
	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑
MASH w/ OTC-ST	56.37	0.64	38.63	53.32	1.10	32.55	46.23	1.64	19.08
Warm-Start Initialization	40.11	0.95	21.97	36.10	0.98	20.04	15.92	0.86	7.80

B. Search Tool Use

In this section, we provide details for GRPO and warm-start training and describe the datasets used for training and evaluation.

B.1. GRPO Training

We use the GRPO implementation of the veRL library (Sheng et al., 2025) for all RL training.

Training hyperparameters For general training hyperparameters, we set the learning rate to 10^{-6} without any warmup or decay and use a gradient clipping norm of 1.0. For policy optimization, we set $\epsilon = 0.2$, entropy coefficient to 0.001, batch size to 64, group size $G = 16$ and perform 1 gradient step per rollout. In early hyperparameter tuning experiments, we observed setting $\beta = 0$ to improve performance, with the associated benefit of freeing the memory used for the reference model. In doing so, we follow other follow-up work on GRPO (Liu et al., 2025).

We perform training for 400 steps and evaluate the model on the task’s validation set every 25 steps. We restrict the use of LLM judges only to the test set and use exact match to estimate accuracy for training and validation. We pick the checkpoint to evaluate using validation tool productivity performance.

Retrieval details We use the retrieval server implementation provided by Search-R1 (Jin et al., 2025) for retrieval. We further follow Search-R1 in masking out tokens from retrieved documents when computing losses. We use the E5 retriever (Wang et al., 2022) with 3 documents returned per query. We enclose each returned query in-between <document> tags.

Inference hyperparameters We perform inference with a temperature of 1.0 during both training and test, and do not use either top- p or top- k sampling. The maximum output length for an individual generation step is 512 tokens and we set the maximum overall output length (with retrieved documents added) to 6144. We truncate outputs exceeding the maximum output length.

Input prompts We use the prompt shown in Figure 3 for tool-use training. This is based on the prompt used by Wang et al. (2025a). For R1 training, on the other hand, we use the prompt shown in Figure 2. This is identical to the R1 prompt used in Search-R1.

B.2. Inference Algorithm

Inference is done according to the procedure detailed in Algorithm 1. Note that this inference procedure during RL training and evaluation is distinct from the structured inference procedure used in warm-start data generation (as described in Algorithm 2). If a model exceeds the maximum number of allowed searches and still attempts a search, it is given a warning message instead. We observed that this did not occur for runs featuring the efficiency reward. Finally, we do not manually append a course-correction message upon failure to generate a properly formatted search or answer tag, as this is a task-specific addition and must be defined for each tool individually.

B.3. Warm-Start

Warm-Start Implementation Details We follow the procedure outlined in Algorithm 2 to construct the warm-start data. We use the Qwen2.5-32B base model as our generator, as it is better capable of following instructions off-the-shelf, but

Algorithm 1 Inference with Multi-Turn Search Tool Calls

Require: Question q , language model π_θ , retriever H

Hyperparameter: Maximum search budget L

Ensure: Trajectory τ

Initialize trajectory $\tau \leftarrow \emptyset$

Initialize action count $l \leftarrow 0$

while $l \leq L + 2$ **do**

Generate action $a_l \sim \pi_\theta(\cdot | q, \tau; H)$ until [`</search>`, `</answer>`, `<eos>`]

Append a_l to trajectory $\tau \leftarrow \tau + a_l$

if `<search>` `</search>` detected in a_l and $l < L$ **then**

Extract search query s_l

Retrieve top- k documents $o_l \sim H(s)$

Append documents to trajectory $\tau \leftarrow \tau + o_l$

else if `<search>` `</search>` detected in a_l **then**

Construct warning message $m = \text{<warning> SEARCH LIMIT REACHED </warning>}$

Append m to trajectory $\tau \leftarrow \tau + m$

else if `<answer>` `</answer>` detected in a_l or `<eos>` detected in a_l **then**

return Final generated response τ

end if

Increment $l \leftarrow l + 1$

end while

return τ

Table 7. Answer accuracy and abstention classification results for the warm-start initialization and the best-performing MASH variant. For answer accuracy, we report Acc over the entire test set and Prec, the accuracy over non-abstained answers. For classification, we report Abs(0), the abstention rate for unanswerable questions (higher better), and Delta, the margin between abstention rates for unanswerable and answerable questions (higher better). The warm-start initialization is poorer than MASH in all cases.

Method	Answer Accuracy						Abstention Classification			
	NaturalQA		HotPotQA		2Wiki		NaturalQA		HotPotQA	
	Acc	Prec	Acc	Prec	Acc	Prec	Abs(0) ↑	Delta ↑	Abs(0) ↑	Delta ↑
MASH w/ OTC-ST	20.9	57.4	17.3	59.9	4.6	32.5	85.4	66.1	91.2	60.3
Warm-Start Initialization	8.28	23.97	8.89	26.14	2.45	6.20	66.17	1.55	68.64	7.69

has not undergone alignment for abstention unlike instruct models. Nonetheless, to ensure that the base model generates properly formatted outputs, we sample 4 candidate outputs for each action and discard the output if it contains unrelated tags or does not add the action ending tag. For think and search actions, we choose a random output. For answer actions, we preferentially choose correct outputs.

Evaluation of trajectories is done with an LLM judge, in this case Qwen2.5-72B-Instruct (Qwen et al., 2025). We follow the same procedure we use to evaluate abstention model outputs, described in Section C.1. If a trajectory is deemed correct, we swap its generated answer with the ground-truth answer for the target dataset to align answers with the dataset format, as we use exact match as the reward.

For a given question q , if we sample $l = 0$ as the target number of actions, we use the prompt used for R1 training (Figure 2) to prevent the model from searching. Otherwise, we use the prompt described in Figure 4.

Training Details We use Huggingface TRL’s SFTTrainer to perform training (von Werra et al., 2020). We use the hyperparameters used by Muennighoff et al. (2025) for performing SFT on reasoning data. Specifically, we use a learning rate of 10^{-5} , weight decay of 10^{-4} , Adam $\beta_1 = 0.9, \beta_2 = 0.95$ and gradient clipping norm of 1. We use a linear learning rate scheduler warmed-up for 5% of training steps and decayed to 0 throughout training. We train for 5 epochs with an effective batch size of 16. As in RL training, tokens corresponding to retrieved documents are masked out from the loss.

Algorithm 2 Warm-Start Trajectory Construction

Require: Datapoint (q, a^*) , generator G , retriever H , maximum searches l_{max} , num samples N

Ensure: Datapoint (q, τ) for SFT

Sample random number of searches $l \sim \{0, \dots, l_{max}\}$

Define $seq \leftarrow [\mathbf{think}, \mathbf{search}] \times l + [\mathbf{think}, \mathbf{answer}]$

for $i = 1 \rightarrow N$ **do**

Initialize current trajectory $\tau_i \leftarrow \emptyset$

for action in seq **do**

Append action start tag $\tau_i \leftarrow \tau_i + \langle \text{action} \rangle$

Generate action $a \sim G(\cdot | q, \tau_i)$ until $\langle /action \rangle$

Append action a to trajectory $\tau_i \leftarrow \tau_i + a$

if action = search **then**

Retrieve top- k documents $o \sim H(a)$

Append o to trajectory $\tau_i \leftarrow \tau_i + o$

end if

end for

end for

Set τ to a random correct τ_i if any, else τ_i with shortest answer.

return τ

Lack of alignment with parametric knowledge In Table 7, we report our warm-start initializations’ performance in terms of the Abs(0) and Delta metrics (as defined in Section 3.2). On both Natural Questions and HotPotQA, the warm-start initialization has miniscule Delta values of 1.55 and 7.69, indicating that the model does not behave differently for unanswerable and answerable questions. Furthermore, as we set $l_{max} = 2$ and choose the target number of searches in warm-start data randomly, two thirds of the data has search (and, therefore, abstention) behavior. This explains the Abs(0) values near 66%.

Task performance of the warm-start initialization We report the performance of the warm-start initializations on the tool-use and abstention settings on Tables 6 and 7 respectively. In both cases, we compare against the best-performing variant of our method, MASH trained with the OTC-Strict reward. For tool-use, the warm-start SFT checkpoints report 16.25 – 30.31% poorer accuracy compared to MASH. For abstention, the warm-start checkpoints similarly severely underperform MASH for all metrics. This provides very strong evidence that the RL component of MASH is crucial for both better search and abstention behaviors.

B.4. Datasets

We run training experiments on three knowledge-intensive datasets – the single-hop dataset Natural Questions (Kwiatkowski et al., 2019), and multi-hop datasets HotPotQA (Yang et al., 2018) and 2WikiMultiHopQA (Ho et al., 2020). Table 8 displays the distribution of question answerability for each dataset. Specifically, each column represents the proportion of test set questions where Qwen2.5-3B-Base achieves an average accuracy of i , estimated using the procedure outlined in Appendix C.1. As seen in the table, the distributions are very similar for Natural Questions and HotPotQA, with 44.72% and 50.11% of questions having an average accuracy of 0 and 11.26% and 7.92% of questions having an average accuracy of 1. 2WikiMultiHopQA is more challenging than both, with 77.26% of questions having an average accuracy of 0 and only 0.85% of questions having average accuracy of 1.

We additionally use the single-hop TriviaQA dataset as part of our out-of-distribution evaluations. For Natural Questions, we use the official splits for training, validation and test. For HotPotQA, 2WikiMultiHopQA and TriviaQA, the official test splits do not contain answers. As a result, we use their official development/validation sets for the purpose of test and construct our own validation sets by sub-sampling from the training set with a 90/10 split. Additionally, as noted in the main text, we filter out the “comparison” and “bridge-comparison” questions from 2WikiMultiHopQA, as these questions are each binary choice questions with heavily skewed answer distributions, causing models to reward hack in practice.

Table 8. Proportion of test set questions where Qwen2.5-3B-Base achieves an average accuracy of i for each $i \in \{0, 0.1, \dots, 0.9, 1\}$ for Natural Questions, HotPotQA and 2WikiMultiHopQA. We find that Natural Questions and HotPotQA have similar distributions of average accuracy, with 2WikiMultiHopQA being harder than both

Method	Percentage of Questions with an Average Accuracy of i											
	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1	
Natural Question	44.72	9.62	6.42	5.06	4.04	3.07	3.65	4.04	3.73	4.40	11.26	
HotPotQA	50.11	9.41	5.91	4.75	3.86	3.94	3.54	3.47	3.71	3.37	7.92	
2WikiMultiHopQA	77.26	8.49	3.77	2.46	1.97	1.33	1.22	0.96	0.85	0.83	0.85	

Input Prompt:

Answer the given question. You should first have a reasoning process in mind and then provides the answer. Show your reasoning in `<think>` `</think>` tags and return the final answer in `<answer>` `</answer>` tags, for example `<answer>` Beijing `</answer>`.
Question: `<question>`

Figure 2. The input prompt used during R1 training experiments. The final `<question>` is replaced by the input question.

Input Prompt:

Answer the given question. You must conduct reasoning between `<think>` and `</think>` every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<search>` query `</search>` and it will return the top searched results between `<document>` and `</document>`. You need to make every search call count and gain helpful results. If you find no further external knowledge is needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. Question: `<question>`

Figure 3. The input prompt used during search tool use experiments. The final `<question>` is replaced by the input question.

Input Prompt:

Answer the given question. You must conduct reasoning between `<think>` and `</think>` every time you get new information. After reasoning, if you find you lack some knowledge, you can ask a question to a search engine by `<search>` query `</search>` and it will return the top searched results between `<document>` and `</document>`. A search query should be an atomic question asking about one, single piece of information.

Example 1:

Question: "Who was born first, Clint Eastwood or Harrison Ford?"

Valid Queries: "`<search>`Clint Eastwood birth date`</search>`" and "`<search>`Harrison Ford birth date`</search>`".

The query "`<search>`Clint Eastwood and Harrison Ford birth date`</search>`" is invalid.

The query "

`<search>`

Clint Eastwood birth date

Harrison Ford birth date

`</search>`"

is also invalid. Do not pack in multiple questions into one query. Each query should be completely independent.

Example 2:

Question: "Which is a genus of palms, Zinnia or Butia?"

Valid Queries: "`<search>`Zinnia genus classification`</search>`" and "`<search>`Butia genus classification`</search>`".

Example 3:

Question: "When did the country where Piltene is located become part of the USSR?"

Initial Query: "`<search>`Piltene location`</search>`"

In each of these examples, you should conduct a search only if you lack the relevant information. Remember, you should decompose questions in your search queries and conduct searches for each atomic question separately. You need to make every search call count and gain helpful results. If you find no further external knowledge is needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`.

Question: `<question>`

Figure 4. The input prompt used when generating tool-use trajectories during warm-start data generation. The final `<question>` is replaced by the input question.

Input Prompt:

Answer the given question. If you are not confident that your answer will be correct, you should abstain from answering by using the phrase 'I am afraid I cannot help you as I do not know the answer to this question.' Question: <question>

Figure 5. The input prompt used during ternary abstention experiments. The final <question> is replaced by the input question.

C. Abstention Experiment Details

In this section, we first detail the pipeline for estimating the average accuracy the base model achieves on each question. This is used to determine both answerability boundaries for abstention training as well as compute abstention classification metrics. We then describe training and inference for our abstention methods.

C.1. Question Accuracy Estimation

We follow the pipeline used by Yang et al. (2024) to estimate the average accuracies. For a given question q , we sample 10 responses $\{\hat{y}_i\}_{i=1}^{10}$ from the untrained model. As all of our experiments are conducted with base models, we perform few-shot prompting. Specifically, for each dataset, we collect correct responses sampled from DeepSeek-V3.1 to 5 questions sampled from the training set and use these as our few-shot examples. For this component, we perform inference with DeepSeek-V3.1 using a temperature of 1 and top- p of 0.8. We likewise perform sampling with Qwen2.5-3B with a temperature of 1, top- p of 0.8 and top- k of 50 to ensure that the base model samples strong outputs and gives a good estimate of knowledge boundaries.

To assess the correctness of a given answer $\hat{y}_i$, we first extract a shortform response and then evaluate the accuracy of this extracted response with an LLM judge. We use DeepSeek-V3.1 in both cases using the few-shot prompts of Yang et al. (2024) (shown in Figures 7 and 8), using greedy decoding for replicability.

C.2. Trained Abstention Model Details

For both the Alignment for Honesty (Yang et al., 2024) and DPO (Rafailov et al., 2023) baselines, we use the exact same training datapoints that MASH was trained on. Furthermore, we perform the exact same number of gradient steps to ensure a fair comparison.

For the Alignment for Honesty variants, we use Huggingface TRL’s SFTTrainer (von Werra et al., 2020). We use a learning rate of 10^{-5} , weight decay of 10^{-4} , Adam $\beta_1 = 0.9$, $\beta_2 = 0.95$ and gradient clipping norm of 1. We use a linear learning rate scheduler warmed-up for 5% of training steps and decayed to 0 throughout training. For the “Absolute” variant of Alignment for Honesty, we use an effective batch size of 64. For the “Multisample” variant, we use an effective batch size of 640 to achieve the same number of gradient steps, as it constructs a datapoint for each question-answer pair sampled during average accuracy estimation.

For the DPO baseline, we use Huggingface TRL’s DPOTrainer. While we take inspiration from Cheng et al. (2024) in constructing the preference dataset, we do not use their two-stage approach featuring an initial SFT stage followed by a DPO stage. Instead, we find that doing DPO training with SFT regularization performs well (Pang et al., 2024) and is more comparable to our other settings. We use the same hyperparameters as in the Absolute variant of Alignment for Honesty. We set the DPO $\beta = 0.1$ and the SFT loss coefficient to 1.

Both models are trained to respond to the prompt shown in Figure 9. We perform inference with a temperature of 1.0, without top- p or top- k sampling, as is done for our MASH models.

C.3. Few-Shot Abstention Prompting Details

For few-shot prompting, we likewise use the prompt shown in Figure 9. As mentioned in Section 3, we average performance over 4 samples. In the case of the few-shot abstention prompt, we use a separate few-shot prompt for each sample. Two of the few-shot prompts feature 3 abstentions on unanswerable questions and 2 answers on always answerable questions. The other two feature 3 answers on always answerable questions and 2 abstentions on unanswerable ones. The answers themselves are sampled from DeepSeek-V3.1.

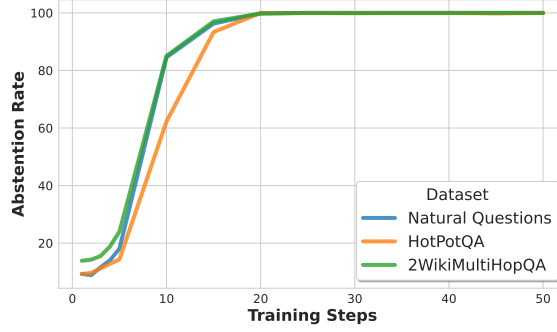


Figure 6. Abstention rate at different training steps when trained with a ternary reward for abstention. Models converge to always abstaining within 25 steps for all datasets.

Input Prompt:

Given a question and a piece of text, if the text does not contain an answer to the question, output “no answer”; otherwise, extract the answer from the text.

Question: What was the last US state to reintroduce alcohol after prohibition?

Text: The last US state to reintroduce alcohol after prohibition was Mississippi. Mississippi legalized alcohol on August 17, 1933, making it the last state to do so.

Output: Mississippi

...

Question: <question>

Text: <model response>

Output:

Figure 7. The input prompt used to extract shortform answers from model outputs during abstention model evaluation and average accuracy estimation for questions.

C.4. Ternary Abstention Baseline Details

The Ternary baseline is trained with GRPO using a ternary reward that rewards correct answers with +1, abstentions with 0 and incorrect answers with −1. We assess correctness using exact match and assess abstention heuristically by checking whether a string from the list [i am afraid, i’m afraid, i cannot help, i can’t help, do not know, don’t know, abstain, abstention] appears as a substring in the final answer. Training uses the prompt shown in Figure 5. Similar to our oracle helper setting, we find that training with this ternary reward leads to models always abstaining within 25 steps. This can be seen in Figure 6.

C.5. Evaluating Abstention Models

The prompt (Figure 7) used for extracting shortform answers by Yang et al. (2024) additionally contains few-shot examples for abstention. As a result, we first determine if a response contains an abstention using this prompt. If it does not contain an abstention, then we evaluate the extracted answer using the prompt in Figure 8.

D. Generalization to Other Model Scales and Families Details

To demonstrate that our insights regarding MASH generalize to models of different scales and families, we conduct further experiments with Qwen2.5-7B-Base and Qwen3-4B-Base respectively. We focus on the HotPotQA dataset due to its mixture of single- and multi-hop questions for these experiments. We conduct RL training under the OTC and MASH w/ OTC-Strict settings and further compare against each abstention baseline, with the exception of Ternary, which trivially learned to abstain on all questions for Qwen2.5-3B-Base. Due to compute limitations, we restrict these experiments to 300 training steps, as opposed to the 400 steps for Qwen2.5-3B-Base. We show results for the first inference setting with search allowed on Tables 9 and 10, and abstention results on Table 11.

Input Prompt:

Please rate the consistency between the reference answer and the proposed answer on a scale of 0 to 1. A rating of 0 indicates inconsistency, while a rating of 1 indicates perfect consistency.

Question: In which country is the Sky Train Rail bridge?

Reference Answer: Canada

Proposed Answer: Thailand

Score: 0

...

Question: <question>

Reference Answer: <gold answer>

Proposed Answer: <extracted answer>

Score:

Figure 8. The input prompt used to evaluate model answers. We follow Yang et al. (2024) in treating an output score higher than 0.7 as indicating correctness.

Input Prompt:

Answer the given question. If you are not confident that your answer will be correct, you should abstain from answering by using the phrase “I am afraid I cannot help you as I do not know the answer to this question.”

Question: <question>

Figure 9. The input prompt used in our abstention models.

Input Prompt:

Answer the given question. You must conduct reasoning between <think> and </think> every time you get new information. After reasoning, if you find you lack some knowledge, you can ask for help by <help> I need help </help> and it will return the answer to the original question between <helper_answer> and </helper_answer>. You need to ask for help only when necessary. If you find no further external knowledge is needed, you can directly provide the answer inside <answer> and </answer>, without detailed illustrations. For example, <answer> Beijing </answer>. Question: <question>

Figure 10. The input prompt used during oracle helper experiments. The final <question> is replaced by the input question.

E. Oracle Helper Details

Implementation details Our oracle helper experiments in Section 4.5 predominantly use the same hyperparameters but differ primarily in prompts and the answer tags used in inference. During GRPO training and during warm-start synthetic data generation when $l = 1$, we use the prompt described in Figure 10. Here, the <search> tag used in normal training becomes a <help> tag and the <document> is replaced by <helper_answer>. Finally, given that the message between the <help> and </help> tags does not matter, we hardcode the specified “I need help” message during warm-start data generation when generating the help action.

Visualization of help-seeking dynamics We find that when trained with the oracle helper, all of our models, regardless of dataset, warm-start procedure or penalty severity, converge to always seeking help. Figure 11 illustrates this for MASH variants on all datasets.

Comparison to the ternary abstention baseline We note that our setting with the oracle helper is equivalent to explicitly training for abstention using RL with ternary rewards. Firstly, as the oracle helper directly returns the correct answer, an efficient policy should perform at most one help call when it performs any. In practice, this results in three possible rewards with decreasing magnitude, with the maximum value given to correct answers, the intermediate value given to help seeking (abstention), and the minimum value given to incorrect answers. This is the same ternary structure explored in the Ternary baseline of Section 3. There we observed a collapse to always abstaining. Here, we observe a collapse to always seeking help, which, under our MASH framework is equivalent to abstention.

Table 9. Accuracy, average number of tool calls (TC) and tool productivity (TP) statistics for OTC and MASH w/ OTC-ST evaluated under the **inference w/ search tools** setting on HotPotQA. MASH w/ OTC-ST continues to outperform the OTC baseline on both Accuracy and TP, achieving a 10% increase on the latter.

Method	Qwen2.5-7B-Base			Qwen3-4B-Base		
	Acc↑	TC↓	TP↑	Acc↑	TC↓	TP↑
OTC (Wang et al., 2025a)	51.52	1.00	25.76	49.11	1.00	24.55
MASH w/ OTC-ST	55.13	1.18	35.37	51.45	0.90	34.13

Table 10. Fine-grained tool use distribution (TC=0/1/2+ search) for baseline OTC and MASH w/ OTC-ST on HotPotQA. We also report answer accuracies for questions in each subset (subscript). TC=0 indicates that the model answers parametrically. MASH can successfully off-load questions to parametric answering (from TC=1 to TC=0) with minimal or no decrease in accuracy.

Method	Qwen2.5-7B-Base			Qwen3-4B-Base		
	0	1	2+	0	1	2+
OTC	0.0 _{0.0}	100.0 _{51.5}	0.0 _{0.0}	0.0 _{0.0}	100.0 _{49.1}	0.0 _{0.0}
MASH w/ OTC-ST	34.6 _{60.7}	31.3 _{61.3}	34.1 _{43.8}	39.2 _{53.5}	31.8 _{56.7}	29.0 _{42.9}

F. Additional Results

F.1. Additional Abstention Metrics

While our analyses on the main paper focused on Abs(0) and Abs(1) as abstention metrics, our trained models show interpretable trends with intermediate values of Abs(0) and Abs(1). We show values for all Abs(i) values for $i \in \{0, 0.1, \dots, 0.9, 1\}$ on Tables 12 and 13. Models’ tendency to abstain decreases as the base model’s average accuracy on a given question increases. We do not include results for 2WikiMultiHopQA as a majority of Abs(i) buckets do not have a high enough support.

F.2. Instruct Model Prompting

As we observed Qwen2.5-3B-Instruct to demonstrate abstention behavior off-the-shelf, we compare the performance of the OTC baseline and MASH w/ OTC-Strict (our best performing variant) to that of the zero- and few-shot prompted Qwen2.5-3B-Instruct model under the abstention setting. For zero-shot prompting, we use the same prompts used in inference for abstention. For few-shot prompting, we re-use the same exemplars used for few-shot prompting with the base models.

Results can be found on 14. We find that MASH w/ OTC-Strict significantly outperforms the zero- and few-shot prompted instruct model. On Natural Questions, MASH w/ OTC-Strict achieves a higher accuracy than the instruct model while maintaining a similar precision value. For HotPotQA and 2WikiMultiHopQA, on the other hand, the instruct model abstains

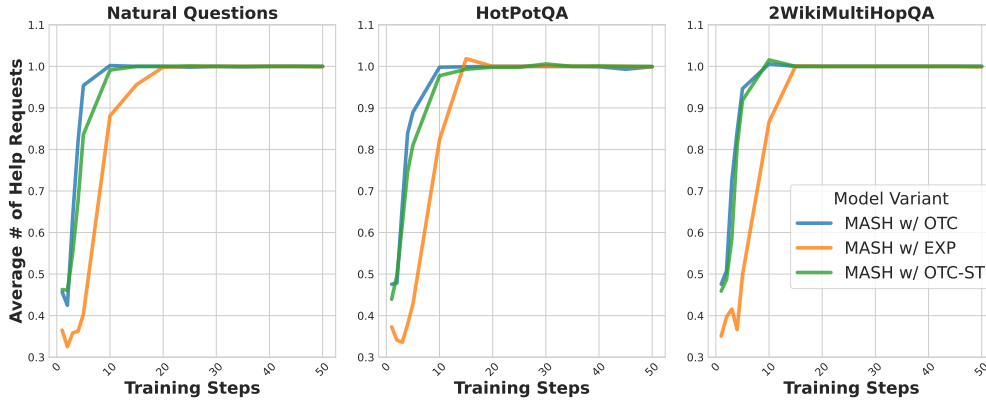


Figure 11. Average number of help requests for all MASH variants when trained with an oracle helper that directly gives away the answer. All variants converge to 1 request within 20 steps.

Table 11. Answer accuracy and abstention classification results for specialized abstention approaches and MASH on HotPotQA. For answer accuracy, we report overall Acc over the entire test set and Prec, i.e. accuracy over the non-abstained answers for each method. For classification, we report Abs(0), i.e. % abstention for unanswerable questions (higher better), and the Delta (higher better) between the % of abstention between unanswerable and answerable questions.

Method	Qwen2.5-7B-Base				Qwen3-4B-Base			
	Acc	Prec	Abs(0) ↑	Delta ↑	Acc	Prec	Abs(0) ↑	Delta ↑
OTC	0.00	—	100.00	0.00	0.00	—	100.00	0.00
MASH w/ OTC-ST	20.98	60.67	87.14	64.51	20.96	53.59	81.63	67.07
5-shot Prompting	23.06	34.17	39.45	24.18	17.58	34.37	59.76	43.72
AFH (Absolute)	25.52	36.73	40.3	27.93	13.44	47.97	82.81	41.5
AFH (Multisample)	17.68	51.36	82.00	53.22	7.25	66.54	96.54	36.51
DPO	24.35	48.25	72.8	60.4	16.72	60.84	92.38	75.87

Table 12. Abs(i) values for each $i \in \{0, 0.1, \dots, 0.9, 1\}$ for specialized abstention approaches and MASH on Natural Questions. We observe that models’ tendency to abstain decreases as the average accuracy for a question increases, with a consistent drop in Abs(i) values from Abs(0) to Abs(1).

Method	Abs(i) for Natural Questions										
	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
OTC	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
MASH w/ OTC	99.86	99.71	100.00	99.86	100.00	99.77	99.81	99.66	99.44	99.53	99.75
MASH w/ OTC-ST	85.45	75.14	67.27	52.85	53.62	51.13	46.40	33.50	27.78	27.37	19.29
MASH w/ EXP	85.65	75.14	66.74	53.55	56.03	50.90	52.26	36.05	32.78	29.25	22.91
5-shot Prompting	60.16	49.86	47.31	38.25	38.45	31.08	34.09	25.34	25.37	21.84	15.60
AFH (Absolute)	67.71	56.30	48.81	40.98	35.52	35.36	35.42	27.05	24.81	20.73	19.66
AFH (Multisample)	87.90	83.41	78.23	69.95	70.69	68.92	60.98	55.65	52.22	46.68	35.81
DPO	84.48	73.48	60.67	51.23	51.03	41.67	40.72	30.31	22.41	17.25	12.90

for a majority of questions, resulting in a very low answer accuracy. MASH w/ OTC-Strict achieves substantially higher accuracies while maintaining precision in these instances.

F.3. Impact on General Task Performance

In order to assess how our training affects the models’ general capabilities, we compare Qwen2.5-3B-Base’s performance against MASH w/ OTC-Strict on separate, general-capability tasks. We use the HotPotQA-trained variant for MASH w/ OTC-Strict. We compare these models on the verifiable instruction-following task of IFEval (Zhou et al., 2023) and the MATH-Hard (Hendrycks et al., 2021) dataset, which features the subset of questions of the MATH dataset with level 5 difficulty. A 4-shot prompt is used in the MATH-Hard setting. We present results in Table 15. We do not observe any degradation in performance following our RL training. In fact, we find that our RL-trained model improves out-of-distribution, with MASH w/ OTC-Strict achieving a 1.60% improvement on IFEval and an 7.68% improvement on MATH-Hard.

These evaluations are done with the commonly used (Gu & Dao, 2024; Touvron et al., 2023; Muennighoff et al., 2025) LM Evaluation Harness of EleutherAI (Gao et al., 2024). We follow the standard task setup for both tasks under the LM Evaluation Harness. We use 250 samples from each subset that is available in these datasets.

G. Out-of-Distribution Results

We present out-of-distribution results for models trained on NaturalQA on Table 16 and for models trained on 2Wiki on Tables 17 and 18. We find that models’ generalization behavior is highly dependent on the dataset they are trained on. For NaturalQA models, DPO achieves superior Abs(0) and Delta, but is outperformed by MASH on TriviaQA. For 2Wiki, on the other hand, where questions are exclusively multi-hop, we find that MASH generalizes reasonably for HotPotQA but struggles on single-hop questions. OTC, on the other hand, performs better in this setting. We note that 2Wiki is highly synthetic and that MASH with OTC-Strict answers parametrically 11.2% more than the OTC baseline on this dataset. We suspect that MASH with OTC-Strict learned dataset-specific shortcuts that hamper its generalization in this process. Nonetheless, with search enabled, all of our help-seeking models outperform DPO, which is ultimately limited to abstention.

Table 13. Abs(i) values for each $i \in \{0, 0.1, \dots, 0.9, 1\}$ for specialized abstention approaches and MASH on HotPotQA. We observe that models’ tendency to abstain decreases as the average accuracy for a question increases, with a consistent drop in Abs(i) values from Abs(0) to Abs(1).

Method	Abs(i) for HotPotQA										
	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
OTC	95.34	89.51	79.09	69.67	60.70	56.08	53.53	55.06	52.18	52.80	53.96
MASH w/ OTC	94.77	87.43	75.00	64.35	52.63	47.95	41.60	45.62	44.09	39.20	42.46
MASH w/ OTC-ST	91.22	82.26	67.44	56.96	46.67	40.67	38.26	40.66	36.55	28.30	30.92
MASH w/ EXP	94.47	87.25	74.71	63.28	52.98	48.03	41.79	46.80	43.27	40.24	41.82
5-shot Prompting	60.54	56.47	49.89	49.08	48.51	45.21	45.42	46.69	44.36	41.20	33.65
AFH (Absolute)	50.42	42.85	37.50	32.46	29.12	27.05	24.43	22.18	21.18	19.50	15.03
AFH (Multisample)	89.20	83.15	75.80	72.09	67.54	63.18	61.55	58.95	58.09	47.80	31.64
DPO	85.91	70.40	56.28	47.80	38.07	33.39	30.06	26.26	22.64	20.70	12.44

Table 14. Answer accuracy (left) and abstention classification (right) results for OTC, MASH w/ OTC-Strict and zero- and five-shot prompting for Qwen2.5-3B-Instruct.

Method	Answer Accuracy						Abstention Classification			
	NaturalQA		HotPotQA		2Wiki		NaturalQA		HotPotQA	
	Acc	Prec	Acc	Prec	Acc	Prec	Abs(0) ↑	Delta ↑	Abs(0) ↑	Delta ↑
OTC	0.0	0.0	12.6	64.5	0.75	24.1	100.0	0.0	95.3	41.4
MASH w/ OTC-ST	20.9	57.4	17.3	59.9	4.6	32.5	85.5	66.2	91.2	60.3
0-shot Abstention	15.7	59.3	2.9	68.0	0.2	34.9	89.3	55.8	98.8	21.2
5-shot Abstention	15.7	58.9	3.7	66.8	0.3	27.8	89.5	55.8	98.5	23.4

H. Compute Requirements and Cost

We perform all experiments on NVIDIA H100 machines. Each individual MASH training experiment takes approximately 100 H100 hours for training and evaluation. In total, we perform 18 full reinforcement learning experiments, leading to approximately 1800 H100 hours. The various abstention experiments are cheaper due to the fact that they do not involve any retrieval, with the Alignment for Honesty Multisample training longest at approximately 4 – 5 hours. Overall, we estimate all training and evaluation experiments taking approximately 1900 H100 hours total. DeepSeek-V3.1 API calls, on the other hand, cost approximately \$400 – 500 total.

Table 15. Performance of Qwen2.5-3B-Base and the HotPotQA-trained MASH w/ OTC-Strict models on IFEval and MATH-Hard. We observe training models to selectively seek help does not degrade general capabilities under our setting.

Method	IFEval	MATH-Hard
Qwen2.5-3B-Base	23.60	15.12
MASH w/ OTC-ST	25.20	22.80

Table 16. Out-of-distribution accuracy (with and without search) and abstention classification results for NaturalQA models. DPO achieves superior Abs(0) and Delta, but is outperformed by MASH on TriviaQA. OTC consistently learns to search on NaturalQA, which generalizes out-of-distribution. However, tool-use enables both OTC and MASH to achieve higher accuracies.

Method	HotPotQA				TriviaQA			
	Acc↑	Acc w/ tool↑	Abs(0) ↑	Delta↑	Acc↑	Acc w/ tool↑	Abs(0) ↑	Delta↑
OTC	0.00	43.05	99.99	-0.01	0.00	72.51	99.99	0.01
MASH w/ OTC-ST	7.62	39.15	93.39	40.66	37.09	65.58	74.44	60.69
DPO	9.1	-	95.66	48.39	34.24	-	84.57	71.45

Table 17. Out-of-distribution accuracy (with and without search) and abstention classification results for 2Wiki models on HotPotQA. DPO achieves superior Abs(0) and Delta, but is outperformed by MASH on Accuracy. For 2Wiki, we find OTC to be more competitive with DPO than MASH on abstention metrics. Nonetheless, tool-use enables both OTC and MASH to achieve higher accuracies.

Method	HotPotQA			
	Acc↑	Acc w/ tool↑	Abs(0) ↑	Delta↑
OTC	4.00	39.86	89.56	14.05
MASH w/ OTC-ST	7.07	39.19	73.36	17.27
DPO	4.07	-	95.43	22.73

Table 18. Out-of-distribution accuracy (with and without search) and abstention classification results for 2Wiki models on single-hop datasets. DPO achieves superior Abs(0), but is outperformed by OTC in terms of Delta and both OTC and MASH in terms of Accuracy. However, we find that MASH struggles at abstention in this setting. Nonetheless, tool-use enables both OTC and MASH to achieve higher accuracies.

Method	Natural Questions				TriviaQA			
	Acc↑	Acc w/ tool↑	Abs(0) ↑	Delta↑	Acc↑	Acc w/ tool↑	Abs(0) ↑	Delta↑
OTC	13.26	39.88	72.85	29.55	24.39	55.37	71.17	33.20
MASH w/ OTC-ST	11.97	33.34	40.28	0.05	23.18	47.42	49.96	19.44
DPO	7.94	-	93.66	28.55	14.71	-	90.05	29.3